



Lesson Plan

Multidimensional Arrays



Multidimensional Array

Topics to be covered:

- Multidimensional Array
- 2D Array
- 2D Vector

Multidimensional Array Introduction

In C++, we can create “array of an array” which are known as a multidimensional array. It stores homogeneous data in a tabular form. Data in multidimensional arrays are stored in row-major order i.e. elements are filled in the current row before moving to the next row.

Syntax to declare an N-Dimensional array:

```
datatype array_name[size1][size2].....[sizeN];
```

A combination of multiple 1D arrays is known as 2D array.

Syntax to declare a 2D array:

```
datatype array_name[rows][columns];
```

where rows imply the number of rows needed for the 2D array and column implies the number of columns needed.

For example:

```
int arr[4][5];
```

Here, arr is a two-dimensional array. It can hold a maximum of 20 elements. Let us understand how.

We can think of this array as a table with 4 rows and each row has 5 columns as shown below.

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

In this array you can store the values as required. Suppose, in the above array you want to store 10 at every index, you can do so using the following code:

```
#include<iostream>
using namespace std;
void main(){
    int arr[4][5];
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 5; j++) {
            arr[i][j] = 10;}
        }
}
```

There are two methods to initialize two-dimensional arrays.

Method 1

```
int arr[2][3]={11,22,33,44,55,66};
```

Method 2

```
int arr[2][3]={11,22,33},{44,55,66};
```

Now that we are equipped with all the relevant information about 2-D arrays, let us move a step ahead to understand 3-D arrays. Although these are rarely used in the common problems that we solve but it is always better to have a slight idea of how the dimensions of an array are scaled up.

Three-dimensional arrays also work in a similar way. For example:

```
double arr[3][2][5];
```

This array arr can hold a maximum of 30 elements of double type.

This array 'arr' can be considered as 3 arrays of 2D which has 2 rows and 5 columns.

Here, we have 5 elements in each row and 2 such rows so total 10 elements in one 2-D array then we have 3 such 2-D arrays so the total number of elements will be 3*10 that is 30.

We can find out the total number of elements in the array simply by multiplying its dimensions:

$3 \times 2 \times 5 = 30$

Taking 2D Array as input

For all implementations, we will have to take inputs from user and work on that data.

Let us learn to take inputs in a 2-D array:

- A 2D array is an array that contains elements in the form of rows and columns. It means we require
- both rows and columns to populate a two-dimensional array. Matrix is the best example of a 2D array.
- We have already learnt to declare 2D arrays and the way to access each element.

Let us have a glance at the code to have a clear idea.

```
#include <iostream>
using namespace std;
int main() {
    int arr[2][3];
    int i, j;
    cout << "\n2D Array Input By user:\n";
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 3; j++) {
            cout << "\ns[" << i << "][" << j << "]= ";
            cin >> arr[i][j];
        }
    }
    cout << "\nThe 2-D Array entered by user is:\n";
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 3; j++) {
            cout << "\t" << arr[i][j];
        }
        cout << endl;
    }
}
```

OUTPUT :

2D Array Input By User :

```
s[0][0] = 2
s[0][1] = 4
s[0][2] = 3
s[1][0] = 6
s[1][1] = 7

s[1][2] = 9
```

The 2-D Array entered by user is :

```
2 4 3
6 7 9
```

Explanation : In the above code firstly we are taking the input of the different elements of the array and after taking input we are printing the elements of the array.

Why do we need Multi-Dimensional Arrays?

- Multi-dimensional arrays are the best choice for representing grids/matrices.
- The most commonly used multidimensional array is the two-dimensional array, also known as a table or matrix.
- The advantage of a multidimensional array is that multi-dimensional input can be taken from the user, with faster access and a predefined size.
- Multidimensional or 2D arrays are easy to access and maintain. You don't have to use multiple variables for each entity which can reside in a single variable
- Throughout your application. Every variable created takes up a specific resource that has to be looked up when accessed.

Introduction to 2D Vectors

A 2D vector is a vector of vectors. Like 2D arrays, values can be declared and assigned to a 2D vector.

A 2D vector is initialized as:

```
vector<vector<datatype>> vector_name
```

Initializing a 2-D vector with help of 1-D vector

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> v1(3, 5); // {5,5,5}
    vector<int> v2(5, 6); // {6,6,6,6,6}
    vector<int> v3(1, 7); // {7}
    vector<vector<int>> vec;
    vec.push_back(v1);
    vec.push_back(v2);
}
```



```
vec.push_back(v3);
for (int i = 0; i < vec.size(); i++) {
    for (int j = 0; j < vec[i].size(); j++) {
        cout << vec[i][j] << " ";
    }
    cout << endl;
}
}
```

Output :

```
5 5 5
6 6 6 6 6
7
```

Initializing a 2-D vector of n rows and m columns

```
vector<vector<datatype>>vector_name(NumOfRows, vector<datatype>(NumOfColumns));
```

For example ,

```
vector<vector<int>>vec(3, vector<int>(4));
```

This statement will create a vector of 3 rows and 4 columns.

Q1. Write a program to store roll number and marks obtained by 4 students side by side in a matrix.

```
#include <iostream>
#include <vector>

int main() {
    const int numStudents = 4;
    const int numSubjects = 2; // Assuming 2 subjects (you can
    change this as needed)

    // Declare a matrix to store roll numbers and marks
    std::vector<std::vector<int>> studentMatrix(numStudents,
    std::vector<int>(numSubjects + 1));
    // Input roll numbers and marks for each student
    for (int i = 0; i < numStudents; ++i) {
        std::cout << "Enter roll number for student " << i + 1 <<
        ": ";
        std::cin >> studentMatrix[i][0]; // Roll number is stored
        in the first column

        for (int j = 1; j <= numSubjects; ++j) {
            std::cout << "Enter marks for subject " << j << " for
            student " << i + 1 << ": ";
            std::cin >> studentMatrix[i][j];
        }
    }
}
```

```
// Display the matrix
std::cout << "\nRoll Number\tSubject 1\tSubject 2\n";
for (int i = 0; i < numStudents; ++i) {
    std::cout << studentMatrix[i][0] << "\t\t";
    for (int j = 1; j ≤ numSubjects; ++j) {
        std::cout << studentMatrix[i][j] << "\t\t";
    }
    std::cout << "\n";
}

return 0;
}
```

Q2. Write a C++ program to find the largest element of a given 2D array of integers.

```
#include <iostream>
#include <climits>

int findLargestElement(int arr[][3], int rows, int cols) {
    int maxElement = INT_MIN;

    // Traverse the 2D array to find the largest element
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            if (arr[i][j] > maxElement) {
                maxElement = arr[i][j];
            }
        }
    }

    return maxElement;
}

int main() {
    const int rows = 3;
    const int cols = 3;
    int myArray[rows][cols];

    // Input elements of the 2D array
    std::cout << "Enter elements of the 2D array:" << std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> myArray[i][j];
        }
    }

    // Find the largest element
    int largestElement = findLargestElement(myArray, rows, cols);
```

```
// Display the result
std::cout << "The largest element in the 2D array is: " <<
largestElement << std::endl;

return 0;
}
```

Q3. Write a program to print sum of all the elements of a 2D matrix.

```
#include <iostream>

int calculateMatrixSum(int arr[][3], int rows, int cols) {
    int sum = 0;

    // Traverse the 2D array to calculate the sum
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            sum += arr[i][j];
        }
    }

    return sum;
}

int main() {
    const int rows = 3;
    const int cols = 3;

    int myArray[rows][cols];

    // Input elements of the 2D array
    std::cout << "Enter elements of the 2D array:" << std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> myArray[i][j];
        }
    }

    // Calculate the sum of elements
    int matrixSum = calculateMatrixSum(myArray, rows, cols);

    // Display the result
    std::cout << "The sum of all elements in the 2D array is: "
<< matrixSum << std::endl;

    return 0;
}
```

Q4. Write a program to add two matrices.

```
#include <iostream>
void addMatrices(const int mat1[][3], const int mat2[][3], int
result[][3], int rows, int cols) {
    // Traverse the matrices and add corresponding elements
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result[i][j] = mat1[i][j] + mat2[i][j];
        }
    }
}

void displayMatrix(const int matrix[][3], int rows, int cols) {
    // Display the elements of the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }
}

int main() {
    const int rows = 3;
    const int cols = 3;

    int matrix1[rows][cols], matrix2[rows][cols],
resultMatrix[rows][cols];

    // Input elements of the first matrix
    std::cout << "Enter elements of the first matrix:" <<
std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> matrix1[i][j];

        }
    }

    // Input elements of the second matrix
    std::cout << "Enter elements of the second matrix:" <<
std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> matrix2[i][j];

        }
    }
}
```

```
// Add the matrices
addMatrices(matrix1, matrix2, resultMatrix, rows, cols);

// Display the result matrix
std::cout << "\nResultant Matrix (Sum of the two matrices):
\n";
displayMatrix(resultMatrix, rows, cols);

return 0;
}
```

Q5. Write a program to print the transpose of the matrix entered by the user and store it in a new matrix.

```
#include <iostream>
void transposeMatrix(const int mat[][3], int transposedMat[][3],
int rows, int cols) {
    // Calculate the transpose of the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            transposedMat[j][i] = mat[i][j];
        }
    }
}

void displayMatrix(const int matrix[][3], int rows, int cols) {
    // Display the elements of the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }
}

int main() {
    const int rows = 3;
    const int cols = 3;

    int originalMatrix[rows][cols], transposedMatrix[cols][rows];

    // Input elements of the matrix
    std::cout << "Enter elements of the matrix:" << std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> originalMatrix[i][j];
        }
    }
}
```

```
// Calculate the transpose of the matrix
transposeMatrix(originalMatrix, transposedMatrix, rows,
cols);

// Display the original matrix
std::cout << "\nOriginal Matrix:\n";
displayMatrix(originalMatrix, rows, cols);

// Display the transposed matrix
std::cout << "\nTransposed Matrix:\n";
displayMatrix(transposedMatrix, cols, rows);

return 0;
}
```

Q6. Write a program to change the given matrix with its transpose.

```
#include <iostream>
void transposeAndModifyMatrix(int mat[][3], int rows, int cols) {
    // Create a temporary matrix to store the transpose
    int transposedMat[cols][rows];

    // Calculate the transpose of the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            transposedMat[j][i] = mat[i][j];
        }
    }

    // Copy the transposed matrix back to the original matrix
    for (int i = 0; i < cols; ++i) {
        for (int j = 0; j < rows; ++j) {
            mat[i][j] = transposedMat[i][j];
        }
    }
}

void displayMatrix(const int matrix[][3], int rows, int cols) {
    // Display the elements of the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }
}

int main() {
    const int rows = 3;
    const int cols = 3;
```

```
int myMatrix[rows][cols];

// Input elements of the matrix
std::cout << "Enter elements of the matrix:" << std::endl;
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
        std::cin >> myMatrix[i][j];
    }
}

// Change the matrix with its transpose
transposeAndModifyMatrix(myMatrix, rows, cols);

// Display the modified matrix
std::cout << "\nModified Matrix (Transposed):\n";
displayMatrix(myMatrix, cols, rows);

return 0;
}
```

Q7. Write a program to rotate the matrix by 90 degrees clockwise.

```
#include <iostream>
void rotateMatrixClockwise(int mat[][3], int rows, int cols) {
    // Transpose the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = i + 1; j < cols; ++j) {
            std::swap(mat[i][j], mat[j][i]);
        }
    }
    // Reverse the order of columns
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols / 2; ++j) {
            std::swap(mat[i][j], mat[i][cols - 1 - j]);
        }
    }
}

void displayMatrix(const int matrix[][3], int rows, int cols) {
    // Display the elements of the matrix
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }
}
```

```
int main() {
    const int rows = 3;
    const int cols = 3;

    int myMatrix[rows][cols];

    // Input elements of the matrix
    std::cout << "Enter elements of the matrix:" << std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> myMatrix[i][j];
        }
    }

    // Rotate the matrix by 90 degrees clockwise
    rotateMatrixClockwise(myMatrix, rows, cols);

    // Display the rotated matrix
    std::cout << "\nRotated Matrix (90 degrees clockwise):\n";
    displayMatrix(myMatrix, rows, cols);

    return 0;
}
```

Q8. Write a program to print the multiplication of two matrices given by the user.

```
#include <iostream>
void multiplyMatrices(const int mat1[][3], const int mat2[][3],
int result[][3], int rows1, int cols1, int rows2, int cols2) {
    if (cols1 != rows2) {
        std::cerr << "Error: Number of columns in the first
matrix must be equal to the number of rows in the second matrix
for multiplication." << std::endl;
        return;
    }

    // Initialize the result matrix with zeros
    for (int i = 0; i < rows1; ++i) {
        for (int j = 0; j < cols2; ++j) {
            result[i][j] = 0;
        }
    }

    // Multiply matrices
    for (int i = 0; i < rows1; ++i) {
        for (int j = 0; j < cols2; ++j) {
            for (int k = 0; k < cols1; ++k) {
                result[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
}
```



```
    }  
    }  
}  
  
void displayMatrix(const int matrix[][3], int rows, int cols) {  
    // Display the elements of the matrix  
    for (int i = 0; i < rows; ++i) {  
        for (int j = 0; j < cols; ++j) {  
            std::cout << matrix[i][j] << " ";  
        }  
        std::cout << std::endl;  
    }  
}  
  
int main() {  
    const int rows1 = 3;  
    const int cols1 = 3;  
  
    const int rows2 = 3;  
    const int cols2 = 3;  
  
    int matrix1[rows1][cols1], matrix2[rows2][cols2],  
    resultMatrix[rows1][cols2];  
  
    // Input elements of the first matrix  
    std::cout << "Enter elements of the first matrix:" <<  
std::endl;  
    for (int i = 0; i < rows1; ++i) {  
        for (int j = 0; j < cols1; ++j) {  
            std::cout << "Element at position (" << i + 1 << ", "  
<< j + 1 << "): ";  
            std::cin >> matrix1[i][j];  
        }  
    }  
  
    // Input elements of the second matrix  
    std::cout << "Enter elements of the second matrix:" <<  
std::endl;  
    for (int i = 0; i < rows2; ++i) {  
        for (int j = 0; j < cols2; ++j) {  
            std::cout << "Element at position (" << i + 1 << ", "  
<< j + 1 << "): ";  
            std::cin >> matrix2[i][j];  
        }  
    }  
  
    // Multiply the matrices  
    multiplyMatrices(matrix1, matrix2, resultMatrix, rows1,  
cols1, rows2, cols2);  
}
```

```
// Display the result matrix
std::cout << "\nResultant Matrix (Multiplication of the two
matrices):\n";
displayMatrix(resultMatrix, rows1, cols2);

return 0;
}
```

Q9. Write a program to print the matrix in wave form.

```
#include <iostream>
void printMatrixInWave(const int mat[][3], int rows, int cols) {
    for (int j = 0; j < cols; ++j) {
        // For even columns, print from top to bottom
        if (j % 2 == 0) {
            for (int i = 0; i < rows; ++i) {
                std::cout << mat[i][j] << " ";
            }
        } else {
            // For odd columns, print from bottom to top
            for (int i = rows - 1; i ≥ 0; --i) {
                std::cout << mat[i][j] << " ";
            }
        }
        std::cout << std::endl;
    }
}

int main() {
    const int rows = 3;
    const int cols = 3;

    int myMatrix[rows][cols];

    // Input elements of the matrix
    std::cout << "Enter elements of the matrix:" << std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> myMatrix[i][j];
        }
    }

    // Display the matrix in wave form
    std::cout << "\nMatrix in Wave Form:\n";
    printMatrixInWave(myMatrix, rows, cols);

    return 0;
}
```

Q10. Write a program to print the matrix in spiral form.

```
#include <iostream>
void printMatrixInSpiral(const int mat[][3], int rows, int cols)
{
    int top = 0, bottom = rows - 1, left = 0, right = cols - 1;

    while (top ≤ bottom && left ≤ right) {
        // Print top row
        for (int i = left; i ≤ right; ++i) {
            std::cout << mat[top][i] << " ";
        }
        top++;

        // Print rightmost column
        for (int i = top; i ≤ bottom; ++i) {
            std::cout << mat[i][right] << " ";
        }
        right--;

        // Print bottom row
        if (top ≤ bottom) {
            for (int i = right; i ≥ left; --i) {
                std::cout << mat[bottom][i] << " ";
            }
            bottom--;
        }
        // Print leftmost column
        if (left ≤ right) {
            for (int i = bottom; i ≥ top; --i) {
                std::cout << mat[i][left] << " ";
            }
            left++;
        }
    }
    std::cout << std::endl;
}

int main() {
    const int rows = 3;
    const int cols = 3;

    int myMatrix[rows][cols];

    // Input elements of the matrix
    std::cout << "Enter elements of the matrix:" << std::endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "Element at position (" << i + 1 << ", "
<< j + 1 << "): ";
            std::cin >> myMatrix[i][j];
        }
    }
}
```

```

    }
}

// Display the matrix in spiral form
std::cout << "\nMatrix in Spiral Form:\n";
printMatrixInSpiral(myMatrix, rows, cols);

return 0;
}

```

Q11. Given an integer 'numRows', generate Pascal's triangle.

```

#include <iostream>
#include <vector>

std::vector<std::vector<int>> generatePascalsTriangle(int
numRows) {
    std::vector<std::vector<int>> triangle;

    for (int i = 0; i < numRows; ++i) {
        std::vector<int> row(i + 1, 1);

        for (int j = 1; j < i; ++j) {
            row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
        }
        triangle.push_back(row);
    }

    return triangle;
}

void displayPascalsTriangle(const std::vector<std::vector<int>>&
triangle) {
    for (const auto& row : triangle) {
        for (int num : row) {
            std::cout << num << " ";
        }
        std::cout << std::endl;
    }
}

int main() {
    int numRows;

    // Input the number of rows for Pascal's Triangle
    std::cout << "Enter the number of rows for Pascal's Triangle:
";
    std::cin >> numRows;
}

```

```
// Generate and display Pascal's Triangle
std::vector<std::vector<int>> pascalsTriangle =
generatePascalsTriangle(numRows);
std::cout << "\nPascal's Triangle:\n";
displayPascalsTriangle(pascalsTriangle);

return 0;
}
```

Q12. Leetcode 861

```
class Solution {
public:
    int matrixScore(vector<vector<int>>& grid) {
        int rows=grid.size();
        int cols=grid[0].size();
        for(int i=0;i<rows;i++){
            if(grid[i][0]==0){
                for(int j=0;j<cols;j++){
                    if(grid[i][j]==0) grid[i][j]=1;
                    else grid[i][j]=0;
                }
            }
        }
        for(int j=0;j<cols;j++){
            int noz=0;
            int no1=0;
            for(int i=0;i<rows;i++){
                if(grid[i][j]==0) noz++;
                else no1++;
            }
            if(noz>no1){
                for(int i=0;i<rows;i++){
                    if(grid[i][j]==0) grid[i][j]=1;
                    else grid[i][j]=0;
                }
            }
        }
        int sum1=0;
        for(int i=0;i<rows;i++){
            int x=1;
            for(int j=cols-1;j>=0;j--){
                sum1=sum1+grid[i][j]*x;
                x=x*2;
            }
        }
        return sum1;
    }
};
```

Q13. Write an efficient algorithm that searches for a value target in an m x n integer matrix which has the following properties :

- **Integers in each row are sorted in ascending from left to right. Integers in each column are sorted in ascending from top to bottom.**

```
class Solution {
public:
    int n;
    bool solve(int i,int j,vector<vector<int>>& mat, int t ){
        while(i<n && j≥0){
            if(mat[i][j]==t){
                return true;
                break;
            }
            else if(mat[i][j]<t) i++;
            else if(mat[i][j]>t) j--;
        }
        return false;
    }
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        n=matrix.size();
        int m=matrix[0].size();

        bool ans=solve(0,m-1,matrix,target);
        if(ans) return true;
        return false;
    }
};
```



**THANK
YOU !**

